

Secure architecture - Netfilter / Iptables

INTRODUCTION

NETFILTER

- modular architecture
- stateful filters
- NAT support
- manages header altering (*packet mangling*)

IPTABLES

interface for **Netfilter**

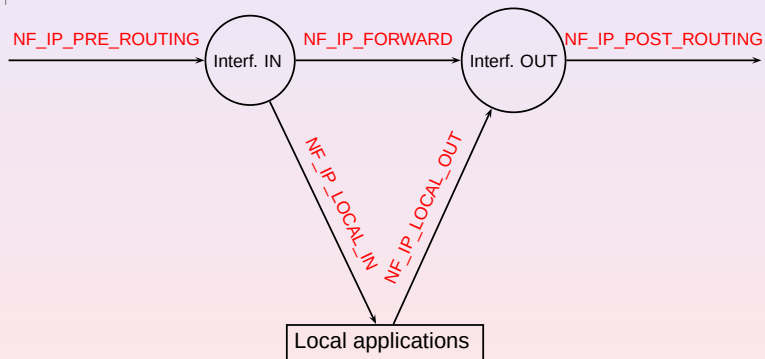
ARCHITECTURE

- offers level-3 **hooks** and **callbacks**
- extra code may be added
- returns a value for packets:
 - NF_ACCEPT accepted, exits current hook
 - NF_DROP refused → packet is deleted
 - NF_STOLEN a fragment will be considered later
 - NF_QUEUE to be processed by user's interface
 - NF_REPEAT calls the hook one more time
 - NF_CONTINUE transferred to the next rule

ARCHITECTURE

- offers level-3 **hooks** and **callbacks**
- extra code may be added
- returns a value for packets:
 - NF_ACCEPT accepted, exits current hook
 - NF_DROP refused → packet is deleted
 - NF_STOLEN a fragment will be considered later
 - NF_QUEUE to be processed by user's interface
 - NF_REPEAT calls the hook one more time
 - NF_CONTINUE transferred to the next rule

IPv4 HOOKS



IP TABLES

- 1 Each packet is sent on a hook.
- 2 It is processed by the tables attached to the hook.

TABLE regroups chains

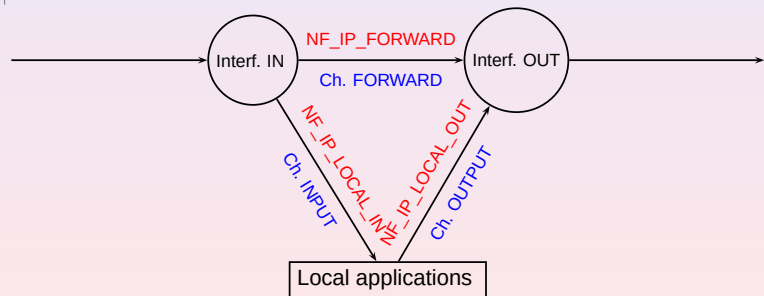
CHAIN linear sequence of rules made of patterns and targets

PATTERN matching criteria

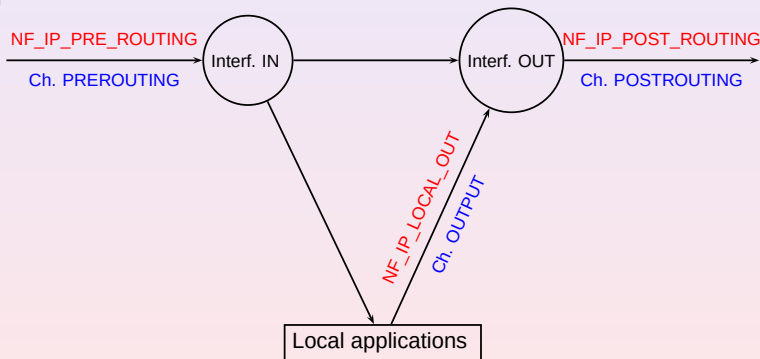
TARGET action if a packet matches a pattern

IP FILTERING: FILTER TABLE

Cibles = ACCEPT, DROP, LOGS, QUEUE (non modified packets)



NAT TABLE



MANGLE TABLE

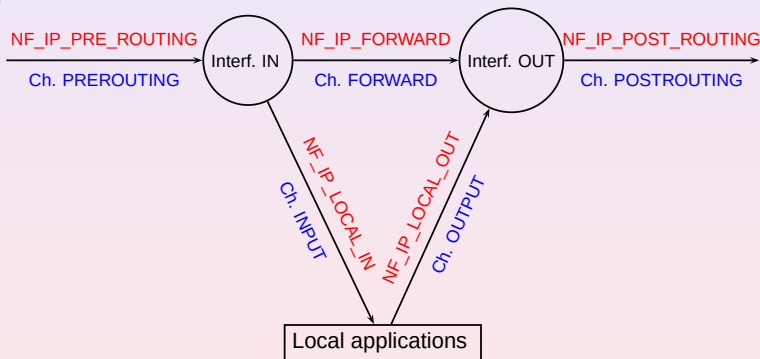
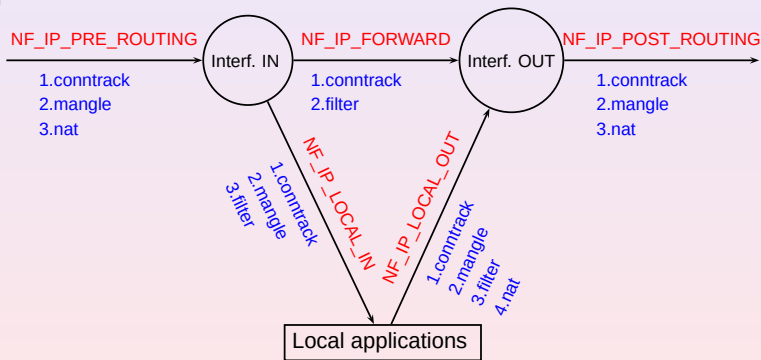


TABLE PROCESSING ORDER



CHAINS

BUILTIN CHAINS

- always present, created with a table containing them
- **default policy**
 - `iptables -P INPUT DROP`
 - `iptables -P FORWARD DROP`
 - `iptables -P OUTPUT DROP`

USER CHAINS

- no default policy
- emulated with an ending rule
- ```
iptables -A MY_CHAIN -j DROP
```

# PATTERNS

- used to assess a packet
- any amount of criteria

```
iptables -A INPUT -s 192.168.10.1 -j ACCEPT
```

```
iptables -A INPUT -p tcp -s 192.168.10.2 --sport 1024: --dport 21 --syn -j ACCEPT
```

```
iptables -A INPUT -m mac --mac-source 00:11:22:33:44:55 -s 192.168.10.2 -j ACCEPT
```

# TARGET

- receives a packet matching a pattern
- indicates the action to realize
- specific targets:
  - *user chain* name → packet exits the current chain and is evaluated in the user chain
  - RETURN → packet exits a *user chain* and goes back into the previous chain
- 0 or 1 target per rule

# TARGET

- receives a packet matching a pattern
- indicates the action to realize
- specific targets:
  - *user chain* name → packet exits the current chain and is evaluated in the user chain
  - RETURN → packet exits a *user chain* and goes back into the previous chain
- 0 or 1 target per rule

# TARGET

- receives a packet matching a pattern
- indicates the action to realize
- specific targets:
  - *user chain* name → packet exits the current chain and is evaluated in the user chain
  - RETURN → packet exits a *user chain* and goes back into the previous chain
- 0 or 1 target per rule

# HOW TO MAKE A RULE?

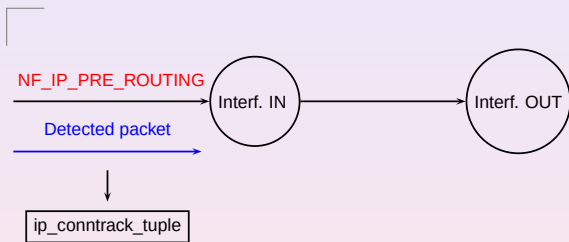
- a rule is made of:
  - ❶ a table (`-t filter`)
  - ❷ a chain (`-A INPUT`)
  - ❸ a pattern (`-p tcp --syn`)
  - ❹ a target (`-j ACCEPT`)
- update a chain:
  - `-A CHAIN [RULE]` : adds RULE at the end of CHAIN
  - `-I CHAIN n [RULE]` : inserts RULE at rank n
  - `-R CHAIN n [RULE]` : replaces RULE at rank n
  - `-D CHAIN n` : deletes rule at rank n



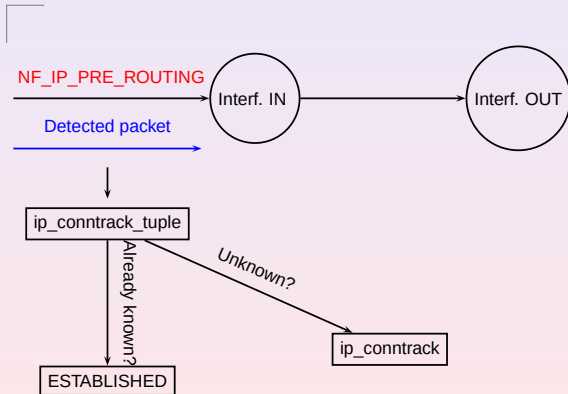
# HOW TO MAKE A RULE?

- a rule is made of:
  - ❶ a table (`-t filter`)
  - ❷ a chain (`-A INPUT`)
  - ❸ a pattern (`-p tcp --syn`)
  - ❹ a target (`-j ACCEPT`)
- update a chain:
  - `-A CHAIN [RULE]` : adds `RULE` at the end of `CHAIN`
  - `-I CHAIN n [RULE]` : inserts `RULE` at rank `n`
  - `-R CHAIN n [RULE]` : replaces `RULE` at rank `n`
  - `-D CHAIN n` : deletes rule at rank `n`

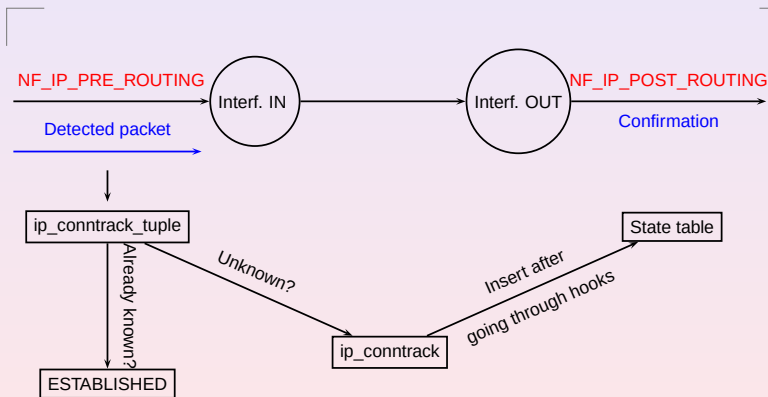
# FOLLOWING STATES 1/3



## FOLLOWING STATES 2/3



## FOLLOWING STATES 3/3



# USER SPACE

## Potential packet states

**NEW** unknown in table of states  $\rightarrow$  new entry in this table if the packet is accepted

**ESTABLISHED** already known in table of states  $\rightarrow \in$  existing session

**RELATED** considered by the session track engine as  $\in$  existing session

**INVALID** undetermined state  $\Rightarrow$  delete packet

# PROTOCOL TRACKING

- Netfilter natively supports:
  - TCP, UDP, ICMP...
  - specific protocols
    - watch some application flux
    - extract information
      - ⇒ build `ip_conntrack` structures
- a packet associated with an existing session gets state **ESTABLISHED** or **RELATED**
- ⇒ ▶ **FTP session**

## NAT: CHANGE SOURCE @

- Hook `NF_IP_POST_ROUTING` before sending packet
  - static target `SNAT` → new source IP@
  - dynamic target `MASQUERADE` : `SNAT` + associate sessions with exit interface's IP@
- interface modified ⇒ current sessions are erased
- ⇒ choose `SNAT` with regards to protocols (e.g. DHCP)

# NAT: CHANGE DESTINATION @

- hooks `NF_IP_PRE_ROUTING` and `NF_IP_LOCAL_OUT` **before** routing
- target `DNAT`: new destination IP@
- target `REDIRECT`: send packets to 127.0.0.1

→ used for transparent proxys

⇒ redirect HTTP traffic to a local proxy (port 3128):

```
iptables -t nat -A PREROUTING -p tcp --dport 80
-j REDIRECT --to-ports 3128
```

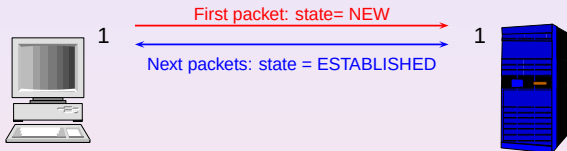


# Figures

# FTP SESSION TRACK 1/6



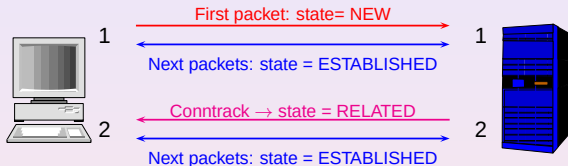
# FTP SESSION TRACK 2/6



```
iptables -A FORWARD -m state --state NEW -p tcp --sport 1024: -d $FTP_SERV --dport 21
-j ACCEPT
```

```
iptables -A FORWARD -m state --state ESTABLISHED -j ACCEPT
```

# FTP SESSION TRACK 3/6

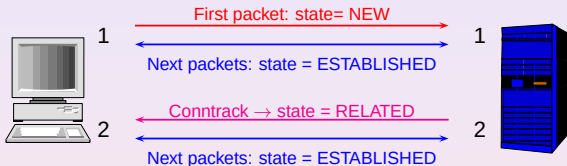


```
iptables -A FORWARD -m state --state NEW -p tcp --sport 1024: -d $FTP_SERV --dport 21
-j ACCEPT
```

```
iptables -A FORWARD -m state --state ESTABLISHED -j ACCEPT
```

```
iptables -A FORWARD -m state --state RELATED -p tcp -s $FTP_SERV --sport 20 --dport 1024:
-j ACCEPT
```

# FTP SESSION TRACK 4/6



```
iptables -A FORWARD -m state --state NEW -p tcp --sport 1024: -d $FTP_SERV --dport 21
-j ACCEPT
```

```
iptables -A FORWARD -m state --state ESTABLISHED -j ACCEPT
```

```
iptables -A FORWARD -m state --state RELATED -p tcp -s $FTP_SERV --sport 20 --dport 1024:
-j ACCEPT
```

```
iptables -A FORWARD -m state --state RELATED -p icmp -j ACCEPT ⇒ ICMP
```

# FTP ACTIVE MODE (5/6)

## PROCEDURE

- 1 Client (open port  $N > 1023$ ): sends **CONNECT** to FTP's server port 21
- 2 Client: starts listening to its port  $N + 1$ : sends **PORT**  $N + 1$  to server
- 3 Server: sends **ACK** to client port  $N + 1$
- 4 Server: connect its port 20 to client's port  $N + 1$
- 5 Client: sends **ACK** to server port 20

⇒ The firewall must accept a connexion from the remote server to the local client!

# FTP PASSIVE MODE (6/6)

[◀ Back](#)

## PROCEDURE

- 1 Client (open port  $N > 1023$ ): sends **CONNECT** to FTP's server port 21
- 2 Client (open port  $N + 1$ ): sends **PASV** command to server port 21
- 3 Server (open port  $P > 1023$ ): sends **PASV** command to client port  $N$
- 4 Client: initiates connection from port  $N + 1$  to server port  $P$
- 5 Server: sends **ACK** to client port  $N + 1$

Warning: not every client supports passive mode!